

ResilientDAG: Mitigating Censorship-Induced Liveness Degradation in Uncertified Leaderless DAG Ledgers through Reactive Witness Injection

Samarth Kothari, Neha Sharma, and K. Hari Babu

Abstract—Leaderless distributed ledgers based on directed acyclic graphs (DAGs) achieve high throughput by allowing every participant to issue blocks concurrently. Their progress, however, depends on participants referencing one another’s blocks honestly. In *uncertified* DAG ledgers—those, such as the IOTA Tangle family, in which parent selection is governed by a tip-selection policy rather than by quorum-validity rules—adversarial participants can selectively refuse to reference blocks issued by honest nodes. Using a discrete-event simulation of an uncertified leaderless DAG, we show that such selective reference censorship fragments the ledger into weakly connected honest and adversarial subgraphs: on a 100-node network with 30% censoring nodes, confirmation throughput falls from 9.5 to 2.1 confirmed rounds per second (a 78% degradation) and confirmation latency exceeds the liveness timeout, while ledger safety remains intact. To restore progress we propose *Backup Pool Injection*, a reactive, protocol-external recovery mechanism in which a small set of passive archival nodes monitors confirmation progress and, upon a liveness timeout, injects *witness blocks* that bridge the fragmented graph. Experiments at network sizes of 100, 500, and 1000 nodes show that the mechanism restores throughput to within 8–9% of the attack-free baseline at a structural overhead of at most 3.2% additional blocks, with overhead decreasing as the network grows. We explicitly scope the threat model: certified DAG-BFT protocols such as DAG-Rider and Bullshark are immune to this attack by construction, because block validity there requires references to $2f+1$ distinct blocks of the preceding round. Our results therefore characterize, and offer a lightweight remedy for, a liveness risk specific to the uncertified setting, where censorship resilience presently relies on tip-selection policy alone.

Index Terms—Distributed ledger, directed acyclic graph, IOTA Tangle, liveness, censorship, tip selection, Byzantine behavior, discrete-event simulation.

I. INTRODUCTION

CONSENSUS is the foundation of trust in distributed ledgers. Classical Byzantine fault-tolerant (BFT) protocols such as PBFT [1] and their modern descendants such as HotStuff [2] provide strong guarantees but serialize block proposal through a leader, which limits throughput. Ledgers structured as directed acyclic graphs (DAGs) remove this bottleneck: every participant issues blocks concurrently, and each block references several earlier blocks, so transaction dissemination proceeds in parallel.

The authors are with the Department of Computer Science and Information Systems, Birla Institute of Technology and Science (BITS) Pilani, Pilani Campus, Rajasthan, India. E-mail: {f20230646, p20240030, khari}@pilani.bits-pilani.ac.in.

DAG ledgers fall into two broad families that differ in how parent references are constrained. In *certified* DAG-BFT protocols such as DAG-Rider [4] and Bullshark [5], a block belonging to round r is valid only if it references at least $2f+1$ distinct blocks of round $r-1$, where f is the Byzantine fault bound. Because an adversary controls at most f nodes, Byzantine participants cannot construct valid blocks that reference only one another; the validity rule itself forces every valid block to incorporate honest history. In *uncertified* leaderless DAG ledgers—exemplified by the IOTA Tangle [6] and its leaderless successor protocol [7], [8]—no such quorum constraint exists. Each new block references a small number of recent tips chosen by a local tip-selection policy, and ledger growth relies on participants applying that policy honestly.

This reliance creates a structural attack surface that has no counterpart in the certified setting. A coalition of adversarial nodes can issue syntactically valid blocks at the normal rate while *selectively refusing to reference* blocks issued by honest nodes, referencing only blocks issued by coalition members. Honest blocks are then starved of approvals: the phenomenon is related to orphanage and lazy-tip behavior studied in the Tangle literature [9], [10], but whereas prior work focuses on detecting such behavior or hardening the tip-selection policy against it, the question of how a ledger can *recover* once fragmentation has already degraded its liveness has received little attention.

This paper studies that recovery problem. We make the following contributions.

- 1) We formalize a *selective reference censorship* attack on uncertified leaderless DAG ledgers and show, through discrete-event simulation, that a static coalition of 30% censoring nodes fragments the ledger and degrades confirmation throughput by 78% without violating safety.
- 2) We propose *Backup Pool Injection*, a reactive recovery mechanism in which passive archival nodes detect liveness stagnation through a timeout and inject *witness blocks* that reference tips in both the honest and the adversarial subgraphs, restoring the connectivity required for confirmation.
- 3) We evaluate the mechanism at network sizes of 100, 500, and 1000 nodes. Throughput recovers to within 8–9% of the attack-free baseline, and the structural overhead of injected blocks does not exceed 3.2% of total ledger size, decreasing with scale.
- 4) We delimit the scope of the result precisely: certified DAG-BFT protocols are immune to this attack by con-

struction, and we discuss what the uncertified setting trades away in exchange for its lighter validity rules.

II. RELATED WORK

A. Leader-Based and Certified DAG BFT

PBFT [1] established practical Byzantine consensus under partial synchrony, and HotStuff [2] reduced its communication cost through leader pipelining; both, however, funnel proposals through a single leader. Narwhal [3] decoupled transaction dissemination from ordering by organizing the mempool as a DAG, and DAG-Rider [4] and Bullshark [5] built full asynchronous and partially synchronous BFT protocols, respectively, on certified round-based DAGs. The defining feature of this family is the quorum-validity rule: a round- r block must reference $2f+1$ blocks of round $r-1$ to be valid at all. As a consequence, the censorship attack studied in this paper cannot arise there—a coalition of at most f nodes can never assemble a valid block from coalition-only parents. The price of this immunity is the machinery of reliable broadcast and certificates, which uncertified ledgers deliberately avoid.

B. Uncertified Leaderless DAG Ledgers

The IOTA Tangle [6] introduced a feeless ledger in which each transaction approves a small number of earlier transactions selected by a tip-selection algorithm, with confirmation determined by accumulated approval weight. The Coordicide project [7] and the Tangle 2.0 consensus [8] removed the central coordinator and made confirmation fully leaderless, which makes honest referencing behavior a correctness assumption rather than a validity rule. Avalanche [11] and Hashgraph [12] occupy adjacent points in this design space, reaching probabilistic or virtual-voting-based agreement over graph structures whose growth is likewise not quorum-certified.

Adversarial referencing behavior in this family has been studied chiefly from the detection and prevention side. Cullen et al. [9] analyze the resilience of DAG ledgers to dishonest tip selection in IoT settings, and Penzkofer et al. [10] detect parasite-chain attacks, in which an attacker grows a private subgraph, from structural anomalies. These works aim to identify misbehavior or to harden the tip-selection policy. Our work is complementary: we accept that fragmentation may occur and ask how the ledger can recover *after* liveness has degraded, without modifying the underlying protocol. To the best of our knowledge, reactive structural repair of a censored uncertified DAG has not been examined before. **[TODO: After your literature pass, add and cite 1–2 further orphanage/tip-selection papers you have actually read, and confirm the novelty claim survives.]**

III. SYSTEM AND ADVERSARY MODEL

A. System Model

We consider a peer-to-peer system of N nodes communicating over an asynchronous network whose message delays are random but finite (eventual delivery). Each node maintains a local copy of a ledger $G = (V, E)$, where vertices are blocks and a directed edge (u, v) records that block u references block

v as a parent. Nodes issue blocks continuously; when issuing a block, a node selects a small set of recent tips of its local view as parents.

Scope of the model. We deliberately study an *uncertified* DAG: parent selection is unconstrained by any quorum-validity rule, as in Tangle-class ledgers [6]–[8]. This choice is what exposes the attack surface studied here. Certified DAG-BFT protocols such as DAG-Rider [4] and Bullshark [5] require every valid round- r block to reference $2f+1$ distinct round- $(r-1)$ blocks; under that rule a minority coalition cannot produce valid blocks that exclude honest history, and the attack below is impossible by construction. Our results therefore apply to the uncertified setting and should not be read as a vulnerability of certified protocols.

For measurement purposes we organize the DAG into logical rounds by structural depth: a block belongs to round r if the majority of its parents belong to round $r-1$. We adopt the standard fault bound

$$f = \left\lfloor \frac{N-1}{3} \right\rfloor, \quad (1)$$

and we say that round r is *confirmed* when it contains at least a quorum of blocks,

$$|V_r| \geq 2f + 1. \quad (2)$$

We emphasize that (2) is a depth-based confirmation heuristic playing the role that cumulative approval weight plays in the Tangle [6]; it is not a BFT commit rule, and we make no claim of agreement finality. It provides a conservative, easily measured proxy for ledger progress: if rounds cannot gather $2f+1$ blocks, weight-based confirmation rules degrade correspondingly.

B. Adversary Model

The adversary statically corrupts up to f nodes ($\approx 30\%$ of N in our experiments). Corrupted nodes issue syntactically valid blocks at the same rate as honest nodes but mount a *selective reference censorship* attack: when selecting parents, they reference only blocks issued by other corrupted nodes, never blocks issued by honest nodes. The adversary does not equivocate, does not delay messages, and does not adapt its strategy in response to recovery actions; we discuss the consequences of these simplifications in Section VIII.

C. Problem Statement

Under this model, honest blocks accumulate references only from other honest blocks, while a parallel adversarial subgraph grows alongside them. Rounds struggle to reach the confirmation quorum (2), so confirmation stalls even though blocks continue to be produced and no safety property is violated. The question we address is whether a *reactive, protocol-external* structural intervention can restore liveness—without modifying the issuance rules followed by ordinary nodes, and at low overhead during both normal operation and recovery.

Algorithm 1 Backup Pool Injection (run at each backup node)

```

1: loop
2:   observe broadcast blocks; update local view  $G = (V, E)$ 
3:   if  $t_{\text{now}} - t_{\text{last confirm}} > \tau$  then
4:     identify tip sets  $T_H$  (honest subgraph) and  $T_A$  (adversarial subgraph)
5:     create witness block  $w$  with parents drawn from  $T_H \cup T_A$ 
6:     broadcast  $w$ 
7:   end if
8:   if a new round confirms then
9:     return to passive monitoring
10:  end if
11: end loop

```

IV. BACKUP POOL INJECTION

Our recovery mechanism augments the network with a small *backup pool* of nodes that are passive in normal operation and act only when liveness stagnates. The design has three elements.

A. Passive Monitoring

Backup nodes receive all broadcast blocks and maintain a complete local view of the DAG, but they do not issue blocks, do not participate in parent selection, and have no influence on confirmation during normal operation. Their steady-state cost is therefore limited to the bandwidth and storage of an archival node. Each backup node tracks the time elapsed since the most recent round confirmation and triggers recovery when

$$t_{\text{now}} - t_{\text{last confirm}} > \tau, \quad (3)$$

where τ is a configurable liveness timeout.

B. Witness Block Injection

Upon activation, a backup node examines its local view, identifies the current tips of the honest and adversarial subgraphs, and issues a *witness block*: a payload-free block whose parent set spans tips of *both* subgraphs. Witness blocks are ordinary, structurally valid blocks; their sole purpose is to re-establish edges between the fragments. Once a witness block propagates, subsequent honest blocks that reference it acquire indirect references into the previously isolated portion of the graph, allowing stalled rounds to reach the confirmation quorum (2) and ledger progress to resume.

C. Deactivation

Activation is self-limiting. When the next round confirms, backup nodes interpret this as restored liveness, cease issuing witness blocks, and return to passive monitoring. This bounds the structural footprint of recovery: per stall episode, only a constant number of witness blocks (one per backup node, with $|B|$ fixed and small) enters the ledger.

D. Design Rationale

Three properties distinguish this mechanism from the alternatives considered in the literature.

Reactive rather than proactive. Approaches that harden the tip-selection policy [9] or continuously screen for anomalous subgraphs [10] impose their cost during normal operation. Backup Pool Injection imposes essentially no cost until the trigger (3) fires, and honest-only executions proceed exactly as in the unmodified system.

Low recovery overhead. Recovery involves neither a voting phase nor a view change. A constant number of witness blocks per stall episode suffices, and ordinary gossip propagation distributes them; in our experiments the cumulative share of injected blocks never exceeds 3.2% of ledger size (Section VI-C).

Structural repair, not protocol redesign. The mechanism does not alter issuance rules, confirmation rules, or any cryptographic primitive of the underlying ledger, and ordinary nodes require no modification. Backup nodes can be operated as auxiliary services by institutional stakeholders, which makes incremental deployment realistic.

V. SIMULATOR DESIGN

We implemented RESILIENTDAG, a discrete-event simulator, in Python 3.9 using SimPy for asynchronous event scheduling and NetworkX for graph maintenance. The implementation is modular: `node.py` runs two concurrent loops per node (block reception and block creation); `network.py` models broadcast with exponentially distributed delays; `dag.py` maintains the graph, performs parent selection, and assigns depth-based rounds; `consensus.py` applies the confirmation rule (2); `attacker.py` implements selective reference censorship; and `backup_pool.py` implements Algorithm 1.

The simulator deliberately abstracts away cryptography: blocks carry lightweight identifiers, and signature creation and verification are not modeled. This isolates the structural dynamics of fragmentation and recovery from cryptographic cost, and permits experiments at a scale of 10^3 nodes; the consequences of this abstraction are discussed in Section VIII. The simulator and all experiment scripts will be made publicly available. **[TODO: Create the public repository (with seeds and plotting scripts) and insert the URL here before submission.]**

VI. EVALUATION

A. Setup

All experiments ran on an Intel Core i7 workstation with 16 GB of RAM under Ubuntu 22.04, using SimPy 4.x and NetworkX 3.x. Network latency is exponentially distributed with a mean of 50 ms; block creation intervals are drawn uniformly from 100–200 ms; the adversary corrupts $\lfloor (N-1)/3 \rfloor$ nodes ($f=33$ for $N=100$). Each configuration is averaged over independent random seeds. **[TODO: Increase from 10 to at least 30 seeds and report mean $\pm$ standard deviation (or 95% confidence intervals) for every cell of Table I.]**

[TODO: Figure 1. Replace the terminal screenshot of the original draft with a proper plot: confirmation latency per round (y-axis, ms) versus round number (x-axis), three curves for Baseline / Attack / Recovered, with the τ timeout shown as a horizontal dashed line. Generate directly from simulator logs with matplotlib; vector PDF, labeled axes, legend. Also investigate why the original log showed Round 4 confirming before Round 1 — determine whether this is an artifact of asynchronous logging or a defect in the confirmation logic, and state the explanation in the text.]

Fig. 1. Per-round confirmation latency under the three scenarios ($N = 100$).

[TODO: Figure 2. Replace the unreadable $N=100$ hairball with a small illustrative DAG ($N \approx 12$): honest blocks in one color, adversarial blocks in another, shown (a) fragmented before witness injection and (b) bridged by a highlighted witness block after injection. Use a layered (round-by-round) layout, include a legend.]

Fig. 2. Structural effect of selective reference censorship and of witness-block injection (illustrative example).

B. Scenarios

We compare three scenarios. **Baseline**: all nodes are honest and select parents from recent tips at random, establishing reference values for confirmation latency, throughput, and ledger growth. **Attack**: 30% of nodes mount selective reference censorship, with no defense present. **Recovered**: the same attack with the Backup Pool Injection mechanism enabled (pool size $|B| = 5$, timeout $\tau = 2000$ ms).

C. Results

Table I summarizes the results. Under honest execution, a 100-node network confirms 9.5 rounds per second at a mean confirmation latency of 110ms. When 30% of nodes censor honest blocks, rounds can no longer assemble $2f+1$ blocks

[TODO: Figure 3. Regenerate the throughput plot cleanly: confirmed rounds per second (y-axis) versus simulation time (x-axis), three curves for the three scenarios, marking the attack onset and the recovery trigger. Remove the embedded duplicate caption of the original; labeled axes, units, legend, vector PDF.]

Fig. 3. Confirmation throughput over time under censorship and recovery ($N = 100$).

TABLE I
RESULTS ACROSS NETWORK SCALES (MEAN OVER SEEDS).

N	Scenario	Latency (ms)	T'put (rds/s)	Overhead (%)
100	Baseline	110	9.5	—
100	Attack	> 2000 (τ)	2.1	—
100	Recovered	140	8.7	3.2
500	Baseline	180	9.2	—
500	Attack	[TODO: run]	[TODO: run]	—
500	Recovered	215	8.5	2.8
1000	Baseline	250	8.8	—
1000	Attack	[TODO: run]	[TODO: run]	—
1000	Recovered	295	8.1	2.4

from connected references: throughput collapses to 2.1 rounds per second—a 78% degradation—and confirmation latency exceeds the 2000 ms liveness timeout (Fig. 1). Blocks continue to be produced throughout, confirming that the attack degrades liveness without affecting safety.

With Backup Pool Injection enabled, the timeout trigger (3) activates the pool, witness blocks bridge the fragments (Fig. 2), and throughput recovers to 8.7 rounds per second at $N=100$, within 8% of the baseline, at a confirmation latency of 140 ms (Fig. 3). The same pattern holds at larger scales: recovered throughput reaches 8.5 and 8.1 rounds per second at $N=500$ and $N=1000$, respectively. Notably, the structural overhead of injected witness blocks *decreases* with network size, from 3.2% of ledger size at $N=100$ to 2.4% at $N=1000$, because a constant number of injected blocks per stall episode is amortized over a larger ledger. Targeted topological repair is thus markedly cheaper than redundancy- or flooding-based alternatives.

[TODO: Add a comparison against at least one baseline defense from the tip-selection-hardening literature (e.g., a restricted/weighted tip-selection rule), so the mechanism is evaluated against an alternative and not only against the undefended attack. This is the single most important addition for journal review.]

VII. DEPLOYMENT CONSIDERATIONS

Several engineering questions arise when moving the mechanism from simulation toward a production ledger.

Pruning. Production ledgers garbage-collect history beyond global finality. The backup pool’s view must respect the pruning horizon so that witness blocks never reference pruned vertices.

Propagation. Gossip dissemination scales linearly in the number of blocks; at thousands of nodes, the burst of witness blocks at activation is small but could be smoothed further with batching or erasure-coded dissemination in the style of Narwhal workers [3].

Pool sizing. A small fixed pool (5–10 nodes) suffices provided each member maintains a complete view. Larger pools reduce detection latency marginally while increasing redundant injections, so witness creation should be coordinated or deduplicated within the pool.

Authentication of witness blocks. In a deployment, witness blocks must be signed, and the pool itself becomes a target: a corrupted backup node could inject misleading bridges or withhold recovery. Requiring a t -of- n threshold signature over each witness block prevents any single corrupted pool member from subverting recovery, at the cost of signature aggregation latency (Ed25519 verification alone costs on the order of 50 μ s per block). Quantifying this cryptographic overhead is part of our planned follow-up work.

Trigger tuning. The static timeout τ must be set above the tail of natural confirmation latency to avoid spurious activations. An adaptive threshold estimated from the local inter-arrival distribution of confirmations would let the pool distinguish censorship from transient network degradation; we regard the static trigger as a proof of mechanism, not a finished design.

VIII. LIMITATIONS

The scope of our claims is bounded by the following limitations. First, the simulator abstracts the ledger to its graph structure: there is no cryptography, no fee or congestion-control logic, and the confirmation rule (2) is a depth-based heuristic rather than a production confirmation rule, so the results are qualitative characterizations of structural liveness behavior rather than formal guarantees. Second, the adversary is static and non-adaptive; an adversary that oscillates between censoring and honest behavior, or that targets the recovery mechanism itself, could interact with the timeout trigger in ways we have not modeled. Third, the trigger threshold τ is heuristic and was not derived from a formal model of network delay. Fourth, our evaluation is empirical; we provide no theoretical bound on recovery time. Finally, our findings apply to uncertified DAG ledgers only; as discussed in Section III, certified DAG-BFT protocols exclude this attack by construction, and we make no claims about them.

IX. CONCLUSION AND FUTURE WORK

We studied selective reference censorship in uncertified leaderless DAG ledgers and showed by simulation that a 30% static coalition that refuses to reference honest blocks degrades

confirmation throughput by 78% while leaving safety intact—a structural liveness risk specific to ledgers whose growth is governed by tip selection rather than quorum-validity rules. We proposed Backup Pool Injection, a reactive mechanism in which passive archival nodes detect stagnation through a timeout and inject payload-free witness blocks that bridge the fragmented graph. Across networks of up to 1000 nodes the mechanism restored throughput to within 8–9% of baseline at no more than 3.2% structural overhead, without modifying the issuance or confirmation rules of the underlying ledger.

Future work will proceed in four directions: replacing the static timeout with an adaptive trigger estimated from observed confirmation inter-arrival times; evaluating adaptive adversaries that alternate between censoring and honest behavior to thrash the trigger; authenticating witness blocks with threshold signatures and measuring the resulting cryptographic overhead; and validating the mechanism against a full implementation of a Tangle-class node rather than a structural simulator.

ACKNOWLEDGMENTS

[TODO: Add funding acknowledgments if applicable.]

REFERENCES

- [1] M. Castro and B. Liskov, “Practical Byzantine fault tolerance,” in *Proc. 3rd USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 1999, pp. 173–186.
- [2] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, “HotStuff: BFT consensus with linearity and responsiveness,” in *Proc. ACM Symp. Principles of Distributed Computing (PODC)*, 2019, pp. 347–356.
- [3] G. Danezis, L. Kokoris-Kogias, A. Sonnino, and A. Spiegelman, “Narwhal and Tusk: A DAG-based mempool and efficient BFT consensus,” in *Proc. 17th European Conf. Computer Systems (EuroSys)*, 2022, pp. 34–50.
- [4] I. Keidar, E. Kokoris-Kogias, O. Naor, and A. Spiegelman, “All you need is DAG,” in *Proc. ACM Symp. Principles of Distributed Computing (PODC)*, 2021, pp. 165–175.
- [5] A. Spiegelman, N. Girdharan, A. Sonnino, and L. Kokoris-Kogias, “Bullshark: DAG BFT protocols made practical,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2022, pp. 2705–2718.
- [6] S. Popov, “The Tangle,” IOTA Foundation, White Paper, 2018.
- [7] S. Popov *et al.*, “The Coordicide,” IOTA Foundation, White Paper, 2020.
- [8] S. Müller, A. Penzkofer, N. Polyanskii, J. Theis, W. Sanders, and H. Moog, “Tangle 2.0: Leaderless Nakamoto consensus on the heaviest DAG,” *IEEE Access*, vol. 10, pp. 105807–105842, 2022.
- [9] A. Cullen, P. Ferraro, C. King, and R. Shorten, “On the resilience of DAG-based distributed ledgers in IoT applications,” *IEEE Internet of Things Journal*, vol. 7, no. 8, pp. 7112–7122, 2020.
- [10] A. Penzkofer, B. Kusmierz, A. Caposelle, W. Sanders, and O. Saa, “Parasite chain detection in the IOTA protocol,” in *Proc. 2nd Int. Conf. Blockchain Economics, Security and Protocols (Tokenomics)*, 2020.
- [11] Team Rocket, M. Yin, K. Sekniqi, R. van Renesse, and E. G. Sirer, “Scalable and probabilistic leaderless BFT consensus through metastability,” *arXiv preprint arXiv:1906.08936*, 2019.
- [12] L. Baird, “The Swirls hashgraph consensus algorithm: Fair, fast, Byzantine fault tolerance,” Swirls, Tech. Rep. SWIRLDS-TR-2016-01, 2016.